

Statistical Methods in Artificial Intelligence

T14.4 – CineMatch India: A Personalised Bollywood and South Indian Movie Recommender

Assignment 3 | IIIT Hyderabad | Academic Year 2025-26

Tech Stack: Python, Streamlit, sentence-transformers (all-MiniLM-L6-v2), scikit-learn, TMDB API, Gemini 1.5 Flash

Team:

- **Hardik Kothari** – 2025201046 – hardik.k@students.iiit.ac.in
- **Gaurav Patel** – 2025201065 – gaurav.patel@students.iiit.ac.in
- **Parv Shah** – 2025201093 – parv.shah@students.iiit.ac.in

Links: GitHub Repo | Live App | Video Demo

1. Problem Statement

Build a working recommendation system for Indian cinema that accepts a movie title, a free-text mood description, or genre/mood chips (or any combination), and returns a ranked list of 10 relevant films with a personalised “why you’ll love this” justification per result.

2. Dataset

Source: TMDB API (free tier). We fetched all movies with original language in {Hindi, Tamil, Telugu, Malayalam, Kannada} and vote_count > 20, yielding **25,000+ titles**.

Fields: id, title, original_title, language, overview, genres, keywords, cast, director, release_date, runtime, popularity, vote_average, vote_count, budget, revenue, production_companies, production_countries, spoken_languages, poster_path, tagline, adult

Key data issue – TMDB popularity and vote_average are both misleading.

TMDB popularity is a proprietary time-decaying score that conflates recent social-media buzz with actual film quality. A 2023 re-release can outrank a critically acclaimed classic purely on recency. TMDB vote_average is unreliable for low-vote films – a film with 3 votes rated 10.0 should not rank above a film with 50,000 votes rated 8.2.

Derived field 1 – Fame Score. Replaces TMDB popularity:

$$\text{FameScore}_i = \sum_{p=1}^3 w_p \cdot \log(1 + \text{AppearanceCount}(\text{cast}_p)) + 0.45 \cdot \log(1 + \text{AppearanceCount}(\text{director}))$$

Billing-position weights: $w_1 = 1.0$, $w_2 = 0.7$, $w_3 = 0.5$. Appearance counts are computed only over films with `vote_count > 50`. Final scores are Min-Max scaled to $[0, 1]$. Rewards established talent rather than trending noise.

Derived field 2 – Bayesian Weighted Rating. Replaces raw `vote_average`:

$$\text{WeightedRating}_i = \frac{v}{v + m} \cdot R + \frac{m}{v + m} \cdot C$$

where v = `vote_count`, R = raw `vote_average`, C = dataset mean rating, m = 60th-percentile vote count. Low-vote films are shrunk toward the mean; high-vote films retain their true rating.

3. System Architecture

Three input modes route into two pipelines:

Input	Pipeline
Movie title only	Semantic Recommender
Free text / mood / genre chips	Hybrid Recommender
Title + text/chips combined	Hybrid Recommender (anchor-blended)

Note on ChromaDB: The public API signature accepts a `collection` parameter (ChromaDB) for retrieval. In our implementation this is unused – all retrieval is done in-memory via `numpy/sklearn`, which is sufficient for 25k titles and avoids the overhead of an external vector store.

3.1 Representation Layer

TF-IDF Soup (plot-first weighting): title x2, overview x4, keywords x2, genres, cast, director. Bigrams, `min_df=2`, 50k features, sublinear TF. Overview is repeated 4x because plot similarity drives user satisfaction more than cast overlap.

Sentence Embeddings: all-MiniLM-L6-v2. Each movie is encoded from “{title}. {overview}”. We tested richer prompts (cast-heavy, genre-heavy) but they degraded title-to-title retrieval – genre/cast tokens dominate the embedding space and mask narrative similarity.

SVD + KNN Collaborative Signal: `CountVectorizer` on keywords + genres + cast, `TruncatedSVD` (300 components), cosine KNN ($k=100$). Lightweight collaborative filter via keyword co-occurrence.

3.2 Anchor Similarity Blend (Title Queries)

The three signals are fused as:

$$s_{\text{anchor}} = 0.35 \cdot s_{\text{tfidf}} + 0.60 \cdot s_{\text{embed}} + 0.05 \cdot s_{\text{cf}}$$

When a free-text query accompanies the anchor title, the embedding component is further mixed:

$$s_{\text{embed}}^* = 0.80 \cdot s_{\text{embed}}^{\text{anchor}} + 0.20 \cdot s_{\text{embed}}^{\text{query}}$$

Raw cosine scores are then converted to **percentile ranks** before weighting. MiniLM scores for Indian films cluster tightly in $[0.7, 0.9]$ (small domain, shared vocabulary) – percentile conversion gives 3x better score spread.

3.3 Cross-Encoder Reranker

After initial ranking, cross-encoder/stsb-roberta-base re-scores the top 80 candidates against a structured query: tone + title + motifs + genres + cast + director + overview. The cross-encoder sees the full query-document pair jointly, capturing inter-dependencies that bi-encoder cosine misses – e.g., separating “revenge drama” from “family drama” when both share similar cast and genre tags. Runtime: ~ 1.2 s on CPU for top-80 pairs.

3.4 Temporal Proximity Score

A Gaussian decay penalises films far from the anchor’s release year:

$$s_{\text{temporal}} = \exp\left(-\frac{(\text{year}_{\text{cand}} - \text{year}_{\text{anchor}})^2}{2\sigma^2}\right), \quad \sigma = 12 \text{ years}$$

This softly prefers era-matched films without hard cutoffs.

3.5 Final Scoring Weights (Semantic Mode)

Component	Weight
Cross-encoder rank	0.24
Plot Jaccard (overview term overlap)	0.20
Anchor similarity rank (blended)	0.10
Director match	0.08
Cast Jaccard	0.08
Embedding rank (direct)	0.08
Keyword Jaccard	0.05
Temporal proximity	0.05
Vote confidence	0.03
Fame Score	0.02
Franchise boost	0.02

Plot Jaccard strips stop-words and generic terms from overviews and computes token overlap – identifies thematic cousins sharing rare motif words (e.g., “betrayal”, “underworld”, “witness”) that embedding cosine alone misses.

Franchise boost normalises title tokens to detect sequels/prequels, ensuring they always surface even with lower vote counts.

3.6 Hybrid Mode (Free Text / Chips)

Genre chips are expanded into **intent prose** before embedding. For example, “Romance” becomes: “A romantic love story with soulful music, emotional chemistry, heartfelt relationships, longing, and falling in love.” Raw chip labels produce weak query vectors; intent prose steers the embedding toward narrative content.

Hybrid final score:

$$\text{Score} = 0.56 \cdot s_{\text{sim}} + 0.14 \cdot s_{\text{chip}} + 0.12 \cdot s_{\text{genre}} + 0.10 \cdot s_{\text{vc}} + 0.05 \cdot s_{\text{fame}} + 0.03 \cdot s_{\text{rating}}$$

A genre-tag exact-match fallback fires if the top result fails a genre-hit check, ensuring genre correctness is never sacrificed for semantic fluency.

3.7 MMR Diversification (Optional)

Maximal Marginal Relevance post-filters the ranked pool to reduce redundancy:

$$\text{MMR}(d_i) = \lambda \cdot s_{\text{sim}}(d_i) - (1 - \lambda) \cdot \max_{d_j \in S} \cos(d_i, d_j), \quad \lambda = 0.7$$

where S is the set of already-selected results. Iteratively selects the candidate that best balances relevance and diversity. Applied when the “Diversify” toggle is on.

3.8 Justification Generation

Per-card justifications use Gemini 1.5 Flash with a strict prompt: one sentence, max 28 words, citing specific metadata, no template phrases. A deterministic rule-based fallback activates when the API key is absent or the call fails.

4. App Interface

Built with Streamlit. Three composable input modes: movie selector dropdown, free-text textarea, genre/mood chip grid (10 genres + 8 moods). Results display as ranked cards with poster, metadata, star rating, match-score bar, and justification. Favourites saved to session state and exportable as CSV. Sidebar: language filter (Hindi / South Indian / custom), decade filter, MMR toggle.

5. Qualitative Results

Important note: This system has no ground-truth evaluation metric. Movie recommendation is inherently subjective – there is no single “correct” output for a given query. We therefore do not report precision/recall. Instead, we evaluate qualitatively: do the results match the intent of the query, and are the outputs coherent and diverse?

Query A: “Intense survival-action movie with relentless enemies and nonstop tension”

1. Pathaan	2. War 2
3. The Return of the Army Man	4. Soch Lo
5. An Action Hero	6. War
7. Holiday	8. Ek Villain
9. Flight	10. Krrish 3

Analysis: Results are correctly dominated by high-octane Hindi action films. “Pathaan”, “War”, and “War 2” are the canonical high-stakes, nonstop-action franchise entries – their top placement confirms the system aligns intent prose with actual film content. “Holiday” and “Ek Villain” are slightly softer but still tension-driven, which reflects the embedding capturing “thriller” semantics even without that word in the query. “Flight” (survival drama) and “Krrish 3” (relentless enemy) match “survival” and “relentless enemies” via plot Jaccard on uncommon motif tokens.

Query B: Pathaan (title-only, semantic mode)

1. War	2. Jawan
3. Fighter	4. Sooryavanshi
5. An Action Hero	6. Attack
7. Jaat	8. Tiger 3
9. War 2	10. Race 2

Analysis: Title-only mode uses the full semantic pipeline with cross-encoder reranking. Top results are precisely the correct peer cluster: YRF spy-universe films (War, Tiger 3, War 2), large-budget patriotic action (Jawan, Fighter, Sooryavanshi), and franchise action (Race 2). This reflects strong director-match and cast-Jaccard signals (Shah Rukh Khan, Hrithik Roshan, Akshay Kumar appearing frequently across these films), combined with cross-encoder correctly scoring “high-octane patriotic thriller” similarity. The franchise boost correctly surfaces War 2 despite it being newer/lower-voted.

6. Key Insights

1. **Percentile ranking over raw cosine is critical.** MiniLM cosines for Indian films cluster in [0.7, 0.9]. Ranking by raw cosine produces near-identical scores; percentile conversion exposes real ordering.
 2. **Intent expansion for chips doubles recall.** Encoding “Romance” as a rich concept description rather than a single word is the single biggest improvement in chip-mode query quality.
 3. **Plot Jaccard catches what embeddings miss.** Two films sharing rare motif words (e.g., “witness”, “underworld”) are often thematic cousins. Jaccard on de-stopworded overview tokens recovers these pairs that MiniLM scores near 0.5.
 4. **Cross-encoder at top-80 is necessary.** Bi-encoder cosine cannot separate thematically similar-but-different films with identical cast/genre metadata. The cross-encoder resolves this at acceptable CPU latency (~1.2s).
 5. **TMDB popularity and raw vote_average actively harm ranking.** Replacing them with Fame Score and Bayesian weighted rating reduced trending-film crowding and surfaced better-rated classics.
-

7. Limitations

- **No user history.** Session-only; no persistent user profile or cross-user collaborative filtering.
 - **Language bias in reranker.** cross-encoder/stsb-roberta-base is English-trained. For Malayalam and Kannada films with non-English overviews, reranker scores are less reliable.
 - **Cold-start for niche films.** Films with few votes and generic overviews score low even if genuinely relevant, since Plot Jaccard and embedding both depend on overview quality.
 - **No quantitative benchmark.** No held-out ground-truth relevance labels; all evaluation is qualitative.
-

8. Conclusion

CineMatch India demonstrates that a well-engineered hybrid of TF-IDF, sentence embeddings, a cross-encoder reranker, and domain-specific signals (Fame Score, Bayesian rating, Plot Jaccard, intent expansion) produces high-quality, explainable recommendations for 25,000+ Indian films across five languages – without any model fine-tuning or labelled data, running under 2 seconds per query on CPU.